

Course Code	ECA1407
Course Name	Embedded Systems
Course Outcome	CO3
Assessment Tool	Pseudocode Writing Test
Question 1 No.	Write pseudocode where one controller task wakes multiple waiting tasks using event flags.
Question 2 No.	Write pseudocode for dynamically creating and deleting sensor tasks in an RTOS.
Question 3 No.	Write pseudocode for a logging task with IDLE, ACTIVE, FLUSHING, and ERROR states using semaphores.
Question 4 No.	Write pseudocode for an alarm system using timers, queues, and event flags together.
Question 5 No.	Write pseudocode for a real-time data acquisition system using timers, semaphores, ISR, and queues.
Maximum Marks	50 (5 × 10)
Bloom's Taxonomy Level	BL4

Evaluation Rubric (10 Marks per Question)

Criteria	Max Marks	Q1	Q2	Q3	Q4	Q5
Understanding of Problem & RTOS Concept	2					
Algorithm Design	3					
Use of Control Structures	2					
Logical Flow & Structure of Pseudocode	1					
Completeness	2					
Total Marks	10	/10	/10	/10	/10	/10

Code of Conduct:

I P.Yaswanth Choudhary (192411136) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Yaswanth Choudhary

QUESTION NO: 22

- Write pseudocode where one controller task wakes multiple waiting tasks using event flags.

PROBLEM UNDERSTANDING:

A controller task monitors system events and, when events occur, it sets the corresponding event flag bits. Multiple tasks wait for specific event flags and are awakened by the controller when their event occurs.

RTOS MECHANISM USED:

Event Flags for task synchronization.

STRUCTURED PSEUDOCODE:

// GLOBAL DEFINITIONS

- DEFINE EVENT FLAG SystemEventFlags
- DEFINE FLAG BIT EV_SENSOR // 0x01
- DEFINE FLAG BIT EV_COMM // 0x02
- DEFINE FLAG BIT EV_DISPLAY // 0x04

EVENT FLAGS:

- EV_SENSOR : New sensor data available
- EV_COMM : New message received
- EV_DISPLAY : Display update required

SYSTEM OVERVIEW



1

PSEUDOCODE (Contd....)

BEGIN

// SYSTEM INITIALIZATION

```
CREATE EVENT FLAG SystemEventFlags
CREATE TASK ControllerTask WITH PRIORITY 1
CREATE TASK SensorTask WITH PRIORITY 2
CREATE TASK CommTask WITH PRIORITY 2
CREATE TASK DisplayTask WITH PRIORITY 2

START SCHEDULER
```

END

TASK: ControllerTask

BEGIN

```
DECLARE Events // local variable
DECLARE NewSensor // boolean
DECLARE NewMessage // boolean
DECLARE DispUpdate // boolean

WHILE TRUE
    // Check system conditions and detect events
    NewSensor = CheckSensorUpdate()
    NewMessage = CheckNewMessage()
    DispUpdate = CheckDisplayUpdate()

    IF (NewSensor) THEN
        SET_EVENT_FLAGS(SystemEventFlags, EV_SENSOR)
    ENDIF

    IF (NewMessage) THEN
        SET_EVENT_FLAGS(SystemEventFlags, EV_COMM)
    ENDIF

    IF (DispUpdate) THEN
        SET_EVENT_FLAGS(SystemEventFlags, EV_DISPLAY)
    ENDIF

    WAIT FOR 10 ms // Controller task period
ENDWHILE

END
```

2

PSEUDOCODE (Contd....)

TASK: SensorTask

BEGIN

WHILE TRUE

```
WAIT_EVENT_FLAGS(SystemEventFlags, EV_SENSOR)
ClearEventBit(SystemEventFlags, EV_SENSOR)
ReadSensorData() // Process new sensor data
ENDWHILE
```

END

TASK: CommTask

BEGIN

WHILE TRUE

```
WAIT_EVENT_FLAGS(SystemEventFlags, EV_COMM)
ClearEventBit(SystemEventFlags, EV_COMM)
ReceiveMessage() // Process received message
ENDWHILE
```

END

TASK: DisplayTask

BEGIN

WHILE TRUE

```
WAIT_EVENT_FLAGS(SystemEventFlags, EV_DISPLAY)
ClearEventBit(SystemEventFlags, EV_DISPLAY)
UpdateDisplay() // Update the display
ENDWHILE
```

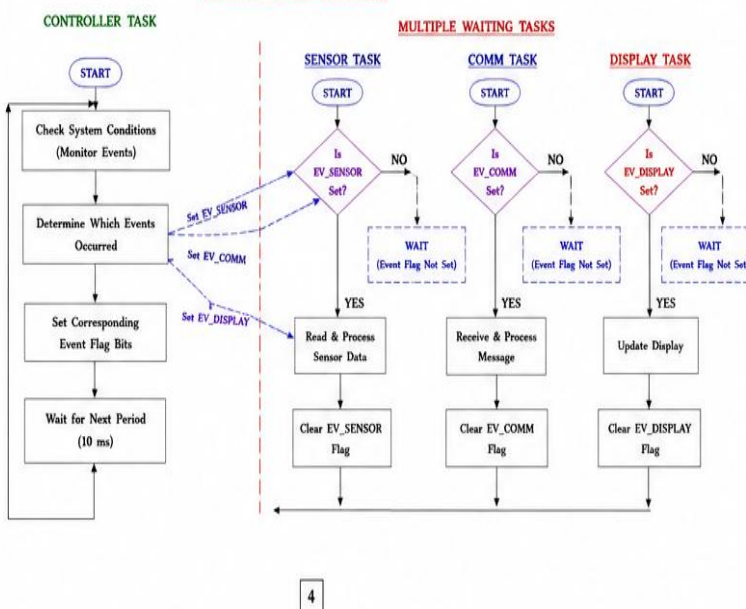
END

QUEUE OPERATIONS SUMMARY

Operation	Performed By	Purpose
SET_EVENT_FLAGS(obj, flag)	ControllerTask	Sets one or more event flag bits
WAIT_EVENT_FLAGS(obj, flag)	Waiting Tasks	Blocks until specified flag bit is set
ClearEventBit(obj, flag)	Waiting Tasks	Clears the specified event flag bit

3

LOGICAL FLOW DIAGRAM



4

EXPLANATION & REAL-TIME EXAMPLE

EXPLANATION:

The ControllerTask monitors various system events and sets the corresponding event flag bits. Each waiting task blocks on its specific event flag. When the controller sets the flag, the corresponding task is unblocked and processes the event. This ensures efficient one-to-many task synchronization using event flags.

KEY POINTS:

- Event flags are used to signal the occurrence of events.
- Multiple tasks wait for different flag bits.
- When a flag bit is set by the controller, the waiting task is awakened.
- This approach decouples task execution and improves system responsiveness.

REAL-TIME EXAMPLE: Smart Home Controller

A smart home controller monitors temperature sensors, motion sensors, and network messages. When events occur, it wakes the respective tasks.



- Sensor Task: Wakes up when temperature or motion sensor data is available.
- Communication Task: Wakes up when a new message (e.g., from mobile app) is received.
- Display Task: Wakes up when the display needs to be updated (e.g., new status to show).

Benefit: One controller task efficiently wakes multiple tasks, ensuring responsive and power-efficient real-time operation.

5

QUESTION NO: 23

- Write pseudocode for dynamically creating and deleting sensor tasks in an RTOS.

PROBLEM UNDERSTANDING:

In an RTOS, sensor tasks may be created when a sensor is connected or enabled, and deleted when it is disconnected or disabled. This saves system resources and allows flexible task management.

RTOS MECHANISM USED:

Dynamic Task Management (Create/Delete Tasks).

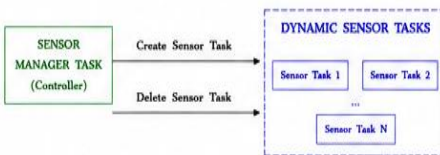
STRUCTURED PSEUDOCODE:

```
// GLOBAL DEFINITIONS
- MAX_SENSORS      = 10
- STACK_SIZE       = 1024
- PRIORITY_SENSOR  = 2
- SENSOR_TASK_HANDLE[MAX_SENSORS]
- SENSOR_CONFIG[MAX_SENSORS] // Stores sensor type, id, etc.
```

SENSOR STATES:

INACTIVE : Sensor not in use
ACTIVE : Sensor task is running

SYSTEM OVERVIEW



1

PSEUDOCODE (Contd....)

```
BEGIN
// SENSOR MANAGER TASK
TASK: SensorManagerTask
BEGIN
    DECLARE INTEGER sensor_id
    DECLARE BOOLEAN create_req, delete_req
    DECLARE SensorConfig config

    WHILE TRUE
        // Check for create or delete requests (from user, events, etc.)
        GetManagerRequests(sensor_id, create_req, delete_req, config)
        IF (create_req == TRUE) THEN
            Call CreateSensorTask(sensor_id, config)
        ENDIF
        IF (delete_req == TRUE) THEN
            Call DeleteSensorTask(sensor_id)
        ENDIF
        // Do other management activities
        WAIT FOR 50 ms // Manager task period
    ENDWHILE
END

FUNCTION: CreateSensorTask(sensor_id, config)
BEGIN
    IF (SENSOR_TASK_HANDLE[sensor_id] == NULL) THEN
        CREATE TASK SensorTask(sensor_id, config)
        WITH STACK SIZE STACK_SIZE
        WITH PRIORITY PRIORITY_SENSOR
        STORE HANDLE IN SENSOR_TASK_HANDLE[sensor_id]
        UPDATE SENSOR_CONFIG[sensor_id] = config
    ELSE
        // Task already exists
    ENDIF
END
```

2

PSEUDOCODE (Contd....)

```
FUNCTION: DeleteSensorTask(sensor_id)
BEGIN
    IF (SENSOR_TASK_HANDLE[sensor_id] != NULL) THEN
        DELETE TASK SENSOR_TASK_HANDLE[sensor_id]
        SET SENSOR_TASK_HANDLE[sensor_id] = NULL
        SET SENSOR_CONFIG[sensor_id].state = INACTIVE
    ELSE
        // Task not found
    ENDIF
END

TASK: SensorTask(sensor_id, config)
BEGIN
    WHILE TRUE
        // Read sensor data
        data = ReadSensor(sensor_id)
        // Process or send data
        ProcessSensorData(sensor_id, data)
        WAIT FOR config.sample_period
    ENDWHILE
END
```

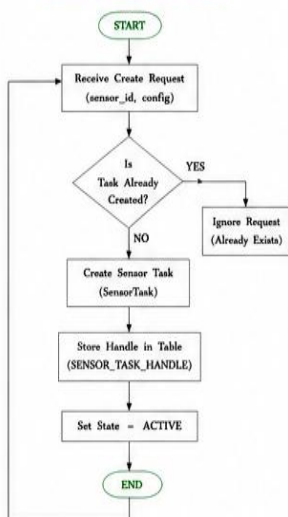
QUEUE OPERATIONS SUMMARY

Operation	Performed By	Purpose
Create Task	Sensor Manager Task	Creates a sensor task dynamically
Delete Task	Sensor Manager Task	Deletes a sensor task dynamically
Read Sensor	Sensor Task	Reads data from the sensor
Process Data	Sensor Task	Processes and/or transmits data

3

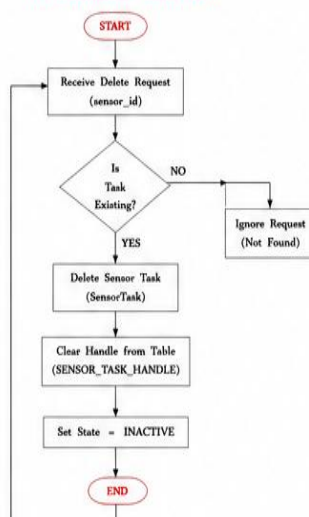
LOGICAL FLOW DIAGRAM

CREATE SENSOR TASK FLOW



4

DELETE SENSOR TASK FLOW



EXPLANATION:

The SensorManagerTask dynamically creates sensor tasks when a sensor is connected or enabled. Each sensor task periodically reads data and processes it. When a sensor is disconnected or disabled, the corresponding task is deleted, releasing system resources. This approach provides flexibility and efficient resource utilization in an RTOS.

KEY POINTS:

- Tasks are created only when needed.
- Tasks are deleted when no longer required.
- Saves memory and CPU resources.
- Improves scalability and system responsiveness.

REAL-TIME EXAMPLE: Industrial IoT (IIoT) Monitoring System

In an industrial IoT system, sensors (temperature, vibration, pressure, etc.) are attached or removed based on machine configuration. The controller dynamically creates a task for each active sensor and deletes it when the sensor is removed.



- New temperature sensor connected → controller creates a sensor task.
- Pressure sensor removed → controller deletes the sensor task.
- System remains efficient and responsive.

Benefit: Dynamic task management ensures resource optimization, scalability, and efficient real-time performance in systems with variable sensors.

5

QUESTION NO: 24

- Write pseudocode for a logging task with IDLE, ACTIVE, FLUSHING, and ERROR states using semaphores.

PROBLEM UNDERSTANDING:

A logging task collects log messages from other tasks using a semaphore signaled when new logs are available. It has four states: IDLE (no logs), ACTIVE (writing logs), FLUSHING (writing remaining logs to storage), and ERROR (I/O failure).

RTOS MECHANISM USED:

Semaphores for signaling log availability and flush requests.

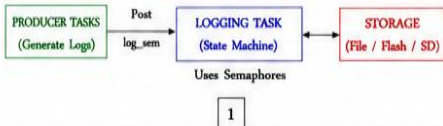
STRUCTURED PSEUDOCODE:

```
// GLOBAL DEFINITIONS
- QUEUE_SIZE           = 50
- LOG_BUFFER[QUEUE_SIZE] // Circular buffer for log messages
- log_head, log_tail    // Queue pointers
- Semaphore log_sem     // Signaled when new log is available
- Semaphore flush_sem   // Signaled to flush remaining logs
- Semaphore mutex_sem   // Mutex for buffer access
```

LOGGING STATES:

- IDLE : No log messages to process
- ACTIVE : Processing available log messages
- FLUSHING : Writing remaining logs to storage
- ERROR : Error occurred during logging

SYSTEM OVERVIEW



PSEUDOCODE (Contd....)

```
BEGIN
// ENUM FOR STATES
IDLE, ACTIVE, FLUSHING, ERROR

// LOGGING TASK
TASK: LoggingTask
BEGIN
DECLARE STATE state = IDLE
DECLARE LogMessage msg
DECLARE INTEGER result

WHILE TRUE
SWITCH (state)
CASE IDLE:
// Wait for a log message or flush request
IF (xSemaphoreTake(log_sem, portMAX_DELAY) == pdTRUE) THEN
state = ACTIVE
ELSE IF (xSemaphoreTake(flush_sem, 0) == pdTRUE) THEN
state = FLUSHING
ENDIF
BREAK
CASE ACTIVE:
// Process available log messages
IF (xQueueReceive(LOG_BUFFER, &msg, 0) == pdPASS) THEN
xSemaphoreTake(mutex_sem, portMAX_DELAY)
result = WriteLogToStorage(msg)
xSemaphoreGive(mutex_sem)
IF (result == SUCCESS) THEN
state = ACTIVE
ELSE
state = ERROR
ENDIF
ELSE
// No more messages, go to IDLE
state = IDLE
ENDIF
BREAK
```

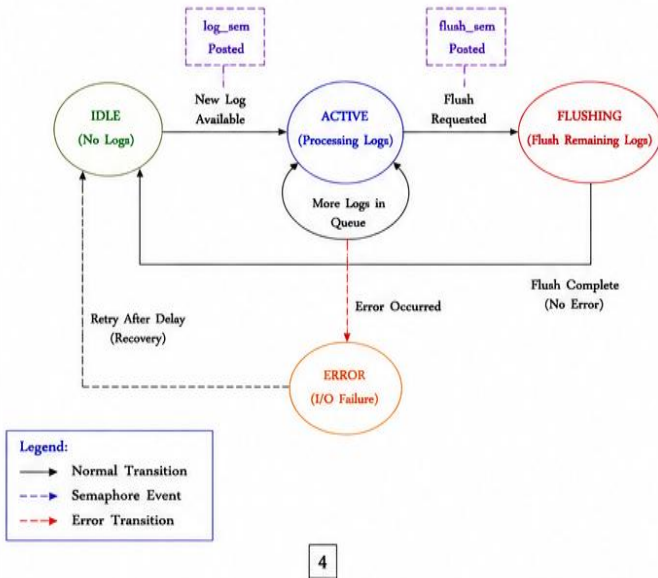
PSEUDOCODE (Contd....)

```
CASE FLUSHING:
// Flush remaining logs to storage
xSemaphoreTake(mutex_sem, portMAX_DELAY)
WHILE (xQueueReceive(LOG_BUFFER, &msg, 0) == pdPASS)
result = WriteLogToStorage(msg)
IF (result != SUCCESS) THEN
state = ERROR
BREAK
ENDIF
ENDWHILE
xSemaphoreGive(mutex_sem)
IF (state != ERROR) THEN
state = IDLE
ENDIF
BREAK
CASE ERROR:
// Handle error (retry or alert)
LogError("Logging error occurred")
NotifySystemError()
Delay(1000) // Wait before retry
state = IDLE
BREAK
ENDSWITCH
ENDWHILE
END
```

QUEUE OPERATIONS SUMMARY

Operation	Semaphore Used	Purpose
Post Log	log_sem	Signal new log message available
Request Flush	flush_sem	Signal to flush remaining logs
Dalfer Access	mutex_sem	Protect buffer and storage access

LOGICAL FLOW DIAGRAM (STATE MACHINE)



EXPLANATION:

The LoggingTask uses semaphores to synchronize with producer tasks and manage log processing. It transitions between IDLE, ACTIVE, FLUSHING, and ERROR states based on semaphore events and operation results.

KEY POINTS:

- log_sem is posted by producer tasks when a new log is available.
- flush_sem is posted to request flushing of remaining logs.
- mutex_sem ensures mutual exclusion while accessing the log buffer and storage.
- ERROR state handles failures and attempts recovery.

REAL-TIME EXAMPLE: Automotive ECU Event Logger

An automotive Electronic Control Unit (ECU) collects events (engine faults, sensor errors, warnings) from multiple modules and logs them to non-volatile memory (eMMC). The logging task ensures reliable storage and can flush logs on demand or during shutdown.



- Normal Run: Logs events as they occur (ACTIVE).
- No Events: Stays in IDLE, waiting for new logs.
- Shutdown Request: Flushes all pending logs to memory (FLUSHING).
- Memory/Write Failure: Enters ERROR state and notifies the system.

Benefit: Ensures reliable logging, data integrity, and robust error handling in safety-critical automotive systems.

QUESTION NO: 25

- Write pseudocode for an alarm system using timers, queues, and event flags together.

PROBLEM UNDERSTANDING:

The alarm system monitors sensors (e.g., door, motion, smoke). When an intrusion or hazard is detected, it raises an alarm. Timers are used for entry delay and alarm duration, queues are used to log events, and event flags are used to signal system conditions between tasks.

RTOS MECHANISM USED:

Timers, Queues, Event Flags.

STRUCTURED PSEUDOCODE:

```
// GLOBAL DEFINITIONS
- QUEUE_SIZE = 50
- EVENT_QUEUE_SIZE = 50
- MAX_LOG_MESSAGE = 128
```

EVENT FLAGS:

- EV_ALARM_ACTIVE (0x01) : Alarm is active
- EV_ENTRY_DELAY (0x02) : Entry delay in progress
- EV_SILENCED (0x04) : Alarm temporarily silenced
- EV_FAULT (0x08) : System fault condition

SYSTEM OVERVIEW



Uses:

- Timers → Entry delay timer, Alarm duration timer
- Queues → Event queue for system events, Log queue for logging
- Event Flags → Signal alarm active, entry delay, silenced, and faults

1

PSEUDOCODE (Contd....)

```
// DATA STRUCTURES
STRUCT Event
    INTEGER type
    INTEGER source_id
    INTEGER timestamp
    CHAR message[MAX_LOG_MESSAGE]
END STRUCT

// QUEUES
QUEUE eventQueue[EVENT_QUEUE_SIZE] // Events from sensors
QUEUE logQueue[QUEUE_SIZE] // Log messages

// TIMERS
TIMER entryDelayTimer // Entry delay
TIMER alarmDurationTimer // Alarm duration

// TASK 1: SENSOR TASK
TASK SensorTask
    BEGIN
        WHILE TRUE
            event = ReadSensors() // Read door, motion, smoke, etc.
            IF event.type != NONE THEN
                Enqueue(eventQueue, event)
                SetEventFlag(EV_NEW_EVENT)
            ENDIF
            WAIT FOR 50 ms
        ENDWHILE
    END

// TASK 2: ALARM MANAGER TASK
TASK AlarmManagerTask
    BEGIN
        WHILE TRUE
            WaitEventFlag(EV_NEW_EVENT)
            ClearEventFlag(EV_NEW_EVENT)
            Dequeue(eventQueue, event)
            ProcessEvent(event)
        ENDWHILE
    END
```

2

PSEUDOCODE (Contd....)

```
PROCEDURE: ProcessEvent(event)
BEGIN
    SWITCH event.type
        CASE INTRUSION_DETECTED:
            IF NOT IsEventFlagSet(EV_ENTRY_DELAY) AND
               NOT IsEventFlagSet(EV_ALARM_ACTIVE) THEN
                SetEventFlag(EV_ENTRY_DELAY)
                StartTimer(entryDelayTimer, ENTRY_DELAY_TIME)
                EnqueueLog("Entry delay started", event)
            ENDIF
            BREAK
        CASE ENTRY_DELAY_TIMEOUT:
            ClearEventFlag(EV_ENTRY_DELAY)
            SetEventFlag(EV_ALARM_ACTIVE)
            StartTimer(alarmDurationTimer, ALARM_DURATION_TIME)
            TriggerAlarmOutput(ON)
            EnqueueLog("Alarm activated", event)
            BREAK
        CASE ALARM_DURATION_TIMEOUT:
            ClearEventFlag(EV_ALARM_ACTIVE)
            TriggerAlarmOutput(OFF)
            EnqueueLog("Alarm stopped", event)
            BREAK
        CASE SILENCE_PRESSED:
            IF IsEventFlagSet(EV_ALARM_ACTIVE) THEN
                SetEventFlag(EV_SILENCED)
                TriggerAlarmOutput(SILENCE)
            ENDIF
            BREAK
        CASE FAULT_DETECTED:
            SetEventFlag(EV_FAULT)
            EnqueueLog("Fault detected", event)
            BREAK
    END SWITCH

// TASK 3: LOGGING TASK
TASK LoggingTask
    BEGIN
        WHILE TRUE
            Dequeue(logQueue, msg)
            WriteToStorage(msg)
        ENDWHILE
    END
```

QUEUE OPERATIONS SUMMARY

Operation	Queue Used	Purpose
Enqueue(event)	eventQueue	Add new event from sensors
Dequeue(event)	eventQueue	Get event for processing
EnqueueLog(msg)	logQueue	Add log message
Dequeue(msg)	logQueue	Get log message to write

TIMER USAGE

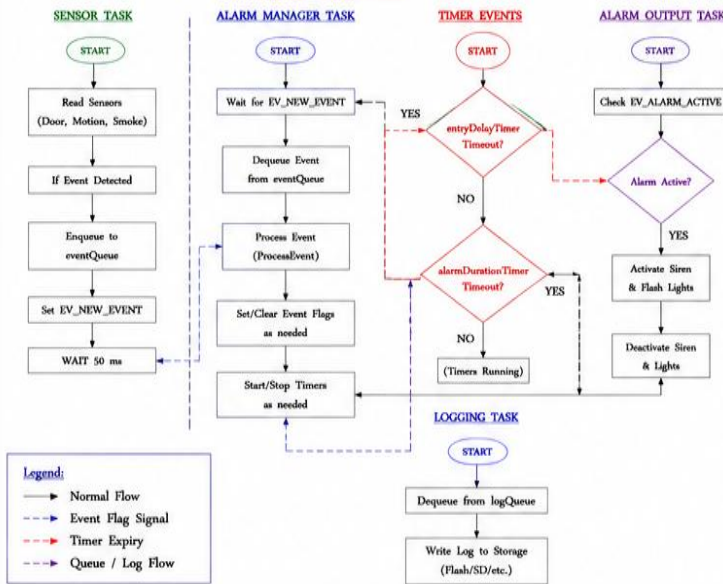
Timer	Purpose	Start By
entryDelayTimer	Entry delay countdown	On intrusion detected
AlarmDurationTimer	Alarm duration countdown	After entry delay timeout

EVENT FLAG USAGE

Flag	Set When	Cleared When
EV_ENTRY_DELAY	Intrusion detected	Entry delay timeout or canceled
EV_ALARM_ACTIVE	Alarm starts	Alarm duration timeout or reset
EV_SILENCED	Silence button pressed	Alarm stopped
EV_FAULT	Fault detected	Fault cleared

3

LOGICAL FLOW DIAGRAM



4

EXPLANATION:

The system uses timers for entry delay and alarm duration, queues for event and log management, and event flags for task synchronization. This ensures reliable alarm operation with proper logging and responsive outputs.

KEY POINTS:

- Sensors post events to eventQueue.
- AlarmManagerTask processes events and controls timers and flags.
- Timers trigger callbacks (events) for state transitions.
- Event flags synchronize tasks and signal system conditions.
- All important activities are logged via logQueue.

REAL-TIME EXAMPLE: Home Security Alarm System

In a home security system, when a door is opened or motion is detected, the system starts an entry delay timer. If the system is not disarmed within the delay, the alarm is activated for a specific duration. All events are logged for audit and review.



- Door/Motion Sensor → Intrusion Detected
- Entry Delay Timer → User has time to disarm
- Alarm Duration Timer → Siren & lights on
- Silence Button → Temporarily silent
- All events → Logged for future review

Benefit: Combining timers, queues, and event flags results in a robust, responsive, and well-synchronized alarm system.

5

QUESTION NO: 26

- Write pseudocode for a real-time data acquisition system using timers, semaphores, ISR, and queues.

PROBLEM UNDERSTANDING:

Acquire data from a sensor periodically in real time. A hardware timer triggers sampling. The ISR reads sensor data and posts it to a queue. A processing task takes data from the queue, timestamps it, and stores or sends it for further analysis. Semaphores are used for synchronization.

RTOS MECHANISM USED:

Timers, Semaphores, ISR, Queues.

STRUCTURED PSEUDOCODE:

// GLOBAL DEFINITIONS

```
- QUEUE_SIZE      = 100
- TIMER_PERIOD_MS = 10    // Sampling every 10 ms
- STACK_SIZE      = 1024
- MAX_LOG_RECORDS = 1000
```

DATA STRUCTURE: SampleData

```
- uint32 timestamp_ms
- int16  sensor_value
```

EVENT FLAGS (Optional)

```
- EV_NEW_DATA (0x01) : New data available
```

SYSTEM OVERVIEW



1

PSEUDOCODE (Contd....)

// GLOBAL OBJECTS

```
QueueHandle_t dataQueue;    // Queue for samples
SemaphoreHandle_t dataSem;   // Semaphore for new data
TimerHandle_t sampleTimer;   // Periodic sampling timer
```

// ISR: Timer Callback (Executed in ISR context)

```
ISR: SampleTimerISR_Callback()
```

BEGIN

```
int16 value = ReadSensor();
SampleData sample;
sample.timestamp_ms = GetSystemTimeMs();
sample.sensor_value = value;
```

```
BaseType_t xHigherPriorityTaskWoken = pdFALSE;
xQueueSendFromISR(dataQueue, &sample, &xHigherPriorityTaskWoken);
xSemaphoreGiveFromISR(dataSem, &xHigherPriorityTaskWoken);
portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
```

END ISR

// TASK: Data Acquisition Task

```
TASK: DAQ_Task()
```

BEGIN

```
SampleData sample;
```

WHILE TRUE

```
// Wait for new data
xSemaphoreTake(dataSem, portMAX_DELAY);

// Read sample from queue
IF (xQueueReceive(dataQueue, &sample, 0) == pdPASS) THEN
    ProcessSample(sample);
    LogSample(sample);
```

ENDIF

ENDWHILE

END TASK

2

PSEUDOCODE (Contd....)

// TASK: Logging/Storage Task (Writes to SD / Sends to Host)

```
TASK: Logger_Task()
```

BEGIN

```
LogRecord record;
```

WHILE TRUE

```
IF (xQueueReceive(logQueue, &record, portMAX_DELAY) == pdPASS) THEN
    WriteToSD(record);    // or SendToHost(record);
```

ENDIF

ENDWHILE

END TASK

QUEUE AND SEMAPHORE SUMMARY

Object	Type	Purpose	Used By
dataQueue	Queue (SampleData)	Holds samples from ISR to DAQ_Task	ISR, DAQ_Task
dataSem	Semaphore (Binary)	Signals new data availability	ISR, DAQ_Task
logQueue	Queue (LogRecord)	Holds processed logs for storage/comm	DAQ_Task, Logger_Task

TIMER SUMMARY

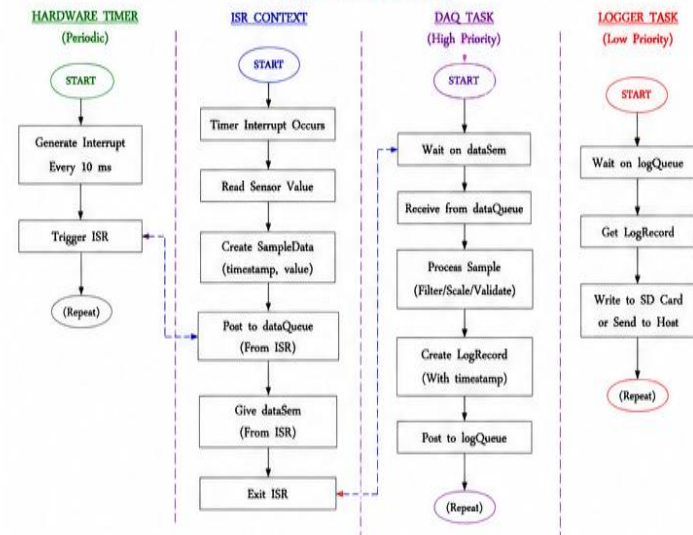
Timer	Type	Period	Purpose	Callback / ISR
sampleTimer	Software / Hardware	10 ms	Trigger periodic sampling	SampleTimerISR_Callback()

INTERRUPT SUMMARY

Interrupt Source	Priority	Purpose	Service Routine
Hardware Timer	High	Triggers periodic sampling	SampleTimerISR_Callback()

3

LOGICAL FLOW DIAGRAM



LEGEND:

→ Normal Flow
--> Interrupt Flow
-.-> Queue / Semaphore Signal

4

EXPLANATION:

The system uses a hardware timer to trigger periodic sampling of sensor data. The ISR reads the sensor quickly and sends the data to a queue, signaling a task via semaphore. The DAQ task processes the data and forwards it to a logger task for storage or transmission. This design ensures minimal ISR latency and reliable real-time data handling.

KEY POINTS:

- Timer ensures precise sampling interval.
- ISR does minimal work: read and post to queue.
- Semaphore synchronizes ISR and DAQ task.
- Queues decouple producer (ISR) and consumer (tasks).
- Logger task handles slow I/O without blocking real-time acquisition.

REAL-TIME EXAMPLE: Industrial Temperature Data Acquisition System

A factory machine has multiple temperature sensors. Data must be sampled every 10 ms and stored on an SD card while also being sent to a monitoring PC.



- Timer (10 ms) → triggers sampling.
- ISR → reads temperature sensor value.
- DAQ Task → filters & validates data.
- Logger Task → writes to SD card and sends to PC.
- Ensures real-time, reliable, loss-free data acquisition.

Benefit: Ensures accurate, timely, and loss-free data acquisition with minimal CPU loading in real-time systems.

5

